

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
Pertemuan Ke-2 dan 3



Disusun oleh:

Nama	NPM
Jaw Syanito Riptanto Putra	2510631170040

PROGRAM STUDI INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS SINGAPERBANGSA KARAWANG
2025/2026

BAB I: PENDAHULUAN

1.1. Landasan Teori

Menurut Herbert Schildt dalam buku “Java: The Complete Reference”, Java merupakan bahasa pemrograman yang sepenuhnya mendukung konsep Object-Oriented Programming (OOP). Dalam OOP, program disusun menggunakan berbagai konsep seperti tipe data, parameter, konstruktor, interface, packages, encapsulation, dan inheritance untuk membangun sistem yang terstruktur dan mudah dikembangkan.

Tipe data dalam Java digunakan untuk menentukan jenis nilai yang dapat disimpan dalam sebuah variabel. Tipe data menentukan ukuran memori yang digunakan serta operasi yang dapat dilakukan terhadap data tersebut. Secara umum, Java memiliki dua jenis tipe data yaitu primitive data type seperti int, double, char, dan boolean, serta non-primitive data type seperti String, array, dan class. Penggunaan tipe data yang tepat sangat penting agar data dapat diproses dengan benar dalam suatu program.

Parameter merupakan variabel yang digunakan dalam deklarasi method untuk menerima nilai dari luar ketika method tersebut dipanggil. Parameter memungkinkan suatu method digunakan untuk memproses berbagai nilai yang berbeda tanpa perlu membuat method baru. Dengan demikian, penggunaan parameter dapat meningkatkan fleksibilitas serta efisiensi dalam penulisan program.

Konstruktor adalah method khusus yang terdapat dalam sebuah class dan digunakan untuk menginisialisasi objek ketika objek tersebut dibuat. Konstruktor memiliki nama yang sama dengan nama class dan tidak memiliki tipe pengembalian (return type). Konstruktor biasanya digunakan untuk memberikan nilai awal pada atribut objek sehingga objek langsung memiliki kondisi tertentu ketika pertama kali dibuat.

Interface merupakan suatu struktur dalam Java yang berisi kumpulan deklarasi method tanpa implementasi. Interface digunakan untuk mendefinisikan perilaku yang harus dimiliki oleh class yang mengimplementasikannya. Dengan adanya interface, Java memungkinkan suatu class untuk memiliki lebih dari satu perilaku melalui implementasi beberapa interface.

Packages dalam Java digunakan untuk mengelompokkan class dan interface ke dalam suatu kelompok atau namespace tertentu. Tujuan penggunaan package adalah untuk mengorganisasi kode program agar lebih rapi, mudah dikelola, serta menghindari konflik penamaan class dalam program yang besar.

Encapsulation merupakan konsep dalam OOP yang digunakan untuk membungkus data dan method dalam satu kesatuan yaitu class serta membatasi akses langsung terhadap data tersebut. Biasanya variabel dalam class dibuat bersifat private dan hanya dapat diakses melalui method tertentu seperti getter dan setter. Dengan demikian, encapsulation membantu menjaga keamanan data dalam program.

Inheritance adalah mekanisme pewarisan sifat dari suatu class ke class lainnya. Class yang mewarisi disebut subclass atau child class, sedangkan class yang diwarisi disebut superclass atau parent class. Melalui inheritance, class turunan dapat menggunakan kembali atribut dan method dari class induk sehingga dapat mengurangi duplikasi kode serta mempermudah pengembangan program.

Berdasarkan uraian tersebut, dapat disimpulkan bahwa konsep tipe data, parameter, konstruktor, interface, packages, encapsulation, dan inheritance merupakan bagian penting dalam pemrograman Java yang mendukung penerapan paradigma Object-Oriented Programming sehingga program dapat dibuat lebih terstruktur, modular, dan mudah dikembangkan.

BAB II: PEMBAHASAN

2.1. Deskripsi Tugas

Membuat program sederhana yang menerapkan konsep Tipe Data, Parameter, Konstruktor, Interface, Packages, Encapsulation DAN Inheritance sesuai instruksi di modul. Instruksi modul :

1. Buatlah Interface bernama `InterfaceEntity`, yang diisi dengan:

- Buat atribut (variable) `public static final double BASE_ATTACK = 300.0`
- Buat atribut (variable) `public static final double BASE_HP = 500.0`
- Buat Method `void infoStatus()`

Selanjutnya buat class `Entity`, yang diisi dengan:

- Buat atribut (variable) `private String nama`
- Buat atribut (variable) `private double hp`
- Buat atribut (variable) `private double attack`
- Buat constructor dengan parameter `-> (String nama, double hp, double attack)`, kemudian isi atribut-atribut yang ada di dalam class `Player` dan untuk bagian `hp` dan `attack` jumlahkan nilainya dengan `BASE_ATTACK` DAN `BASE_HP`
- Buat method `void infoStatus`, untuk menampilkan semua status

Selanjutnya buat class `GameRPG`, dan jalankan class `player`, sebagai contoh:

```
Entity player = new Entity("Himmel", 100, 250)
player.infoStatus()
```

2. Buat class turunan dari class `Entity` dengan nama `Knight`, yang diisi dengan:

- Buat atribut (variable) `private double defense;`
- Buat constructor dengan parameter yang sama dengan class `Entity` dan tambahkan parameter `-> (double defense)`. Gunakan keywords `super` untuk mengirimkan datanya ke class `Entity`
- Buat method `infoStatus` lagi untuk menampilkan status dari `Knight`. Tips, gunakan `super.infoStatus()`.

Selanjutnya buat object baru lagi untuk menjalankan class `Knight`, sebagai contoh:

```
Knight knight = new Knight("Stark", 200, 150, 100)
knight.infoStatus()
```

2.2. Hasil dan Analisis

Hasil Eksekusi Program (Output)

```
run:
Nama   : Marsha
Hp      : 700.0
Attack : 900.0

Nama   : Sentinel
Hp      : 1000.0
Attack : 700.0
Defense: 200.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

Gambar 1. Hasil Eksekusi Program GameRPG.java

Link Repository GitHub : [https://github.com/jawsyanito/Pemrograman-Berbasis-Objek_25040/tree/e10f1ab59fd95d3ab1fb86d7c35107f108e9e4dd/Pertemuan%203%20\(Packages%2C%20Encapsulation%2C%20dan%20Inheritance%20\)/Tugas%202%20%26%203](https://github.com/jawsyanito/Pemrograman-Berbasis-Objek_25040/tree/e10f1ab59fd95d3ab1fb86d7c35107f108e9e4dd/Pertemuan%203%20(Packages%2C%20Encapsulation%2C%20dan%20Inheritance%20)/Tugas%202%20%26%203)

Analisis

Saya membuat program berbasis Java yang terdiri dari beberapa bagian utama, yaitu InterfaceEntity, class Entity, class Knight, dan class GameRPG. Program ini bertujuan untuk memahami konsep Pemrograman Berorientasi Objek (OOP) seperti penggunaan interface, class, constructor, inheritance (pewarisan), serta penggunaan method untuk menampilkan status karakter dalam sebuah permainan RPG sederhana.

Pada bagian pertama, saya membuat sebuah interface bernama InterfaceEntity. Di dalam interface ini terdapat dua atribut bertipe double yaitu BASE_ATTACK dengan nilai 300.0 dan BASE_HP dengan nilai 500.0. Kedua atribut ini bersifat public static final sehingga nilainya tetap dan dapat digunakan oleh class lain yang mengimplementasikan interface tersebut. Selain itu, saya juga membuat sebuah method bernama infoStatus() yang nantinya akan digunakan untuk menampilkan status dari suatu entity di dalam permainan.

```
public interface InterfaceEntity {
    public static final double BASE_ATTACK = 300.0;
    public static final double BASE_HP = 500.0;
    public void infoStatus();
}
```

Selanjutnya saya membuat class Entity yang mengimplementasikan InterfaceEntity. Pada class ini saya mendeklarasikan tiga variabel yaitu nama bertipe String, serta hp dan attack yang bertipe double. Ketiga variabel tersebut bersifat private sehingga hanya dapat diakses di dalam class tersebut. Kemudian saya membuat sebuah constructor dengan parameter (String nama, double hp, double attack) yang digunakan untuk mengisi nilai atribut pada saat objek dibuat. Di dalam constructor tersebut, nilai hp dan

attack dijumlahkan dengan nilai BASE_HP dan BASE_ATTACK yang berasal dari interface. Hal ini bertujuan agar setiap entity memiliki nilai dasar HP dan attack tambahan.

```
public class Entity implements InterfaceEntity {
    private String nama;
    private double hp;
    private double attack;

    public Entity (String nama, double hp, double attack){
        this.nama = nama;
        this.hp = hp + BASE_HP;
        this.attack = attack + BASE_ATTACK;
    }
}
```

Setelah itu saya membuat method infoStatus() yang berfungsi untuk menampilkan seluruh status dari entity, yaitu nama, jumlah HP, dan nilai attack. Method ini menggunakan System.out.println() untuk menampilkan informasi tersebut ke layar ketika program dijalankan.

```
public void infoStatus() {
    System.out.println("Nama : " + nama);
    System.out.println("Hp : " + hp);
    System.out.println("Attack : " + attack);
}
```

Kemudian saya membuat class turunan bernama Knight yang merupakan turunan dari class Entity. Pada class ini saya menambahkan satu variabel baru yaitu defense bertipe double yang digunakan untuk menyimpan nilai pertahanan karakter Knight. Selanjutnya saya membuat constructor dengan parameter (String nama, double hp, double attack, double defense). Pada constructor ini digunakan keyword super() untuk mengirimkan nilai nama, hp, dan attack ke constructor milik class Entity.

```
public class Knight extends Entity {
    private double defense;

    public Knight(String nama, double hp, double attack, double defense){
        super(nama, hp, attack);
        this.defense = defense;
    }
}
```

Pada class Knight saya juga membuat kembali method infoStatus() untuk menampilkan informasi tambahan yaitu nilai defense. Di dalam method tersebut saya menggunakan super.infoStatus() untuk memanggil method infoStatus() milik class Entity, sehingga data nama, HP, dan attack tetap ditampilkan. Setelah itu, saya menambahkan perintah System.out.println() untuk menampilkan nilai defense milik Knight.

Terakhir, saya membuat class GameRPG sebagai class utama yang berisi method main() untuk menjalankan program. Di dalam method main() saya membuat sebuah objek dari class Entity dengan nama player, kemudian mengisi data karakter namanya Marsha dan memanggil method infoStatus() untuk menampilkan statusnya. Selanjutnya saya membuat keluaran " " untuk spasi saja agar output terlihat lebih rapih. Setelah itu saya juga membuat objek dari class Knight dengan nama knight, kemudian mengisi datanya dengan nama Sentinel, lalu memanggil method infoStatus() untuk menampilkan seluruh status karakter Knight termasuk nilai defense.

```
public class GameRPG {  
    public static void main (String[] args){  
        Entity player = new Entity("Marsha",200,600);  
        player.infoStatus();  
  
        System.out.println("          ");  
  
        Knight knight = new Knight("Sentinel",500,400,200);  
        knight.infoStatus();  
    }  
}
```

BAB III: KESIMPULAN

Pada praktikum ini saya berhasil memahami dan menerapkan beberapa konsep dasar Pemrograman Berorientasi Objek (OOP) dalam Java, seperti penggunaan interface, class, object, atribut, constructor, dan method. Saya dapat membuat interface yang berisi konstanta dan method, kemudian mengimplementasikannya pada class untuk menyimpan serta menampilkan data status sebuah entity.

Saya juga memahami bagaimana constructor digunakan untuk menginisialisasi nilai atribut ketika objek dibuat, serta bagaimana nilai tersebut dapat dimodifikasi dengan menambahkan nilai dasar yang berasal dari interface. Selain itu, saya mempelajari konsep inheritance (pewarisan) dengan membuat class turunan yang dapat mewarisi atribut dan method dari class induk.

Melalui praktikum ini saya juga memahami penggunaan keyword super untuk memanggil constructor dan method dari class induk, serta konsep method overriding untuk menambahkan atau menyesuaikan fungsi method pada class turunan. Praktikum ini menjadi dasar penting untuk memahami bagaimana struktur program Java dapat dibuat lebih terorganisir dan mudah dikembangkan menggunakan konsep OOP.

Link Repository GitHub : [https://github.com/jawsyanito/Pemrograman-Berbasis-Objek_25040/tree/e10f1ab59fd95d3ab1fb86d7c35107f108e9e4dd/Pertemuan%203%20\(Packages%2C%20Encapsulation%2C%20dan%20Inheritance%20\)/Tugas%202%20%26%203](https://github.com/jawsyanito/Pemrograman-Berbasis-Objek_25040/tree/e10f1ab59fd95d3ab1fb86d7c35107f108e9e4dd/Pertemuan%203%20(Packages%2C%20Encapsulation%2C%20dan%20Inheritance%20)/Tugas%202%20%26%203)

LEMBAR PENILAIAN DAN MASUKAN

(Bagian ini akan diisi oleh Asisten Laboratorium)

Kriteria Penilaian	Nilai
Kebenaran Kode & Kesesuaian Output (40%)	
Analisis & Pemahaman (40%)	
Format Laporan (20%)	
Total Nilai	

Masukan/Saran dari Asisten Laboratorium:

(Bagian ini akan diisi oleh Asisten Laboratorium)

--